

Exp No :2 DEVELOPING AND DEPLOYING A SIMPLE CHAINCODE**Aim:**

To develop, package, install, approve, commit, and execute a simple asset transfer chaincode in a **Hyperledger Fabric test network** using the **JavaScript programming language**.

Requirements:

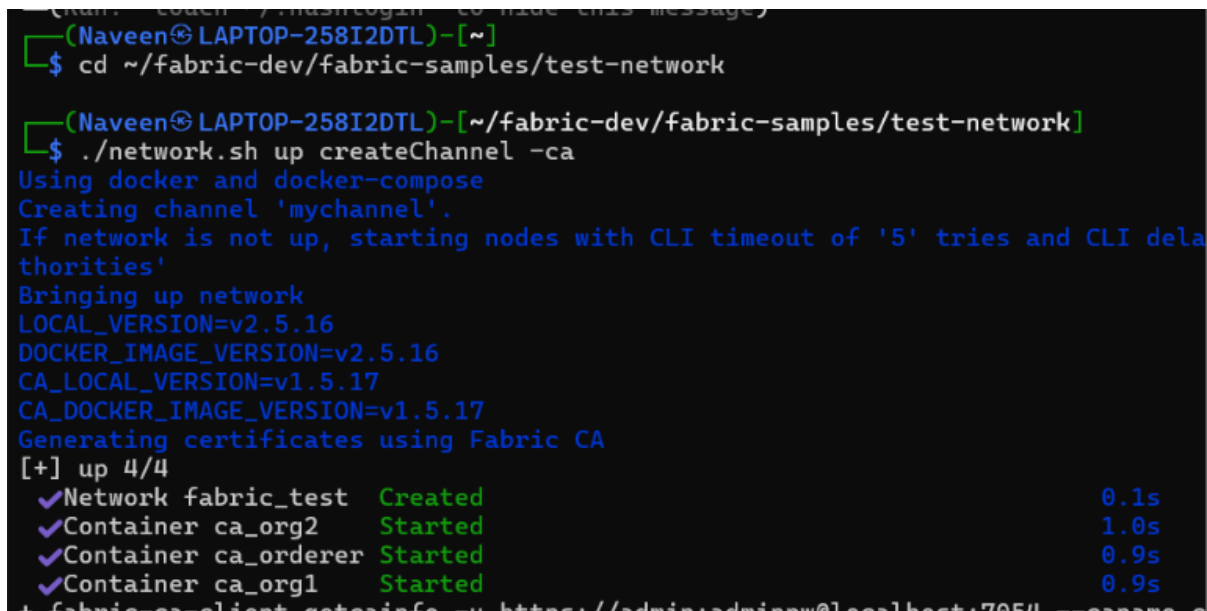
- Linux Operating System (Kali)
- Docker and Docker Compose
- Go Programming Language
- Git and Curl
- Hyperledger Fabric Samples, Binaries and Docker Images

Procedure:**1. Navigate to the Test Network**

```
cd ~/fabric-dev/fabric-samples/test-network
```

2.Start the Fabric Network

```
./network.sh up createChannel -ca
```



```
(Naveen@LAPTOP-258I2DTL)-[~]  
$ cd ~/fabric-dev/fabric-samples/test-network  
  
(Naveen@LAPTOP-258I2DTL)-[~/fabric-dev/fabric-samples/test-network]  
$ ./network.sh up createChannel -ca  
Using docker and docker-compose  
Creating channel 'mychannel'.  
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay  
authorities'  
Bringing up network  
LOCAL_VERSION=v2.5.16  
DOCKER_IMAGE_VERSION=v2.5.16  
CA_LOCAL_VERSION=v1.5.17  
CA_DOCKER_IMAGE_VERSION=v1.5.17  
Generating certificates using Fabric CA  
[+] up 4/4  
✓Network fabric_test Created 0.1s  
✓Container ca_org2 Started 1.0s  
✓Container ca_orderer Started 0.9s  
✓Container ca_org1 Started 0.9s  
+ fabric-ca-client getcainfo -u https://admin:adminpw@localhost:7054 --cacame
```

3. Deploy the JavaScript Chaincode

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-  
basic/chaincode-javascript
```

```

(Naveen@LAPTOP-258I2DTL)~/fabric-dev/fabric-samples/test-network
$ ./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ grep '^basic_1.0:1d4d445573112813889f87c307bc2b5f5f664c87b24c035bee15cbcc7f59b629$'
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz
+ res=0
2026-09-01 20:23:01.309 IST 0001 INFO [cli.lifecycle.chaincode] submitInstallProposal -> Installed rem
445573112813889f87c307bc2b5f5f664c87b24c035bee15cbcc7f59b629\022\tbasic_1.0" >
2026-09-01 20:23:01.310 IST 0002 INFO [cli.lifecycle.chaincode] submitInstallProposal -> Chaincode cod
c307bc2b5f5f664c87b24c035bee15cbcc7f59b629
Chaincode is installed on peer0.org1

```

4. Verify Installed Chaincode

peer lifecycle chaincode queryinstalled

```

(Naveen@LAPTOP-258I2DTL)~/fabric-dev/fabric-samples/test-network
$ peer lifecycle chaincode queryinstalled
Installed chaincodes on peer:
Package ID: basic_1.0:1d4d445573112813889f87c307bc2b5f5f664c87b24c035bee15cbcc7f59b629, Label: basic_1.0

```

5. Initialize the Ledger

peer chaincode invoke -o localhost:7050 \

--ordererTLSHostnameOverride orderer.example.com \

--tls --cafile "\$ORDERER_CA" \

-C mychannel -n basic \

-c '{"function":"InitLedger","Args":[]}'

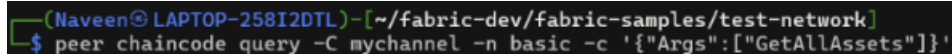
```

(Naveen@LAPTOP-258I2DTL)~/fabric-dev/fabric-samples/test-network
$ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile "$ORDERER_CA" -C mychannel -n basic -c '{"functi
on":"InitLedger","Args":[]}'
2026-09-01 20:34:20.249 IST 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200

```

6. Query All Assets

```
peer chaincode query -C mychannel -n basic \
-c '{"Args":["GetAllAssets"]}'
```



```
(Naveen@LAPTOP-258I2DTL)~/fabric-dev/fabric-samples/test-network
$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```



```
(Naveen@LAPTOP-258I2DTL)~/fabric-dev/fabric-samples/asset-transfer-basic/chaincode-javascript
$ sed -n '16,35p' lib/assetTransfer.js
async InitLedger(ctx) {
  const assets = [
    {
      ID: 'asset1',
      Color: 'blue',
      Size: 5,
      Owner: 'Tomoko',
      AppraisedValue: 300,
    },
    {
      ID: 'asset2',
      Color: 'red',
      Size: 5,
      Owner: 'Brad',
      AppraisedValue: 400,
    },
    {
      ID: 'asset3',
      Color: 'green',
      Size: 10,
```

7. Create a New Asset

```
peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls --cafile "$ORDERER_CA" \
-C mychannel -n basic \
-c '{"function":"CreateAsset","Args":["asset10","Black","15","Arun","800"]}'
```

8. Query the Newly Created Asset

```
peer chaincode query -C mychannel -n basic \
-c '{"Args":["ReadAsset","asset10"]}'
```

9. Transfer Asset Ownership

```
peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
```

```
--tls --cafile "$ORDERER_CA" \

-C mychannel -n basic \

-c '{"function":"TransferAsset","Args":["asset10","David"]}'
```

10. Verify Updated Asset

```
peer chaincode query -C mychannel -n basic \

-c '{"Args":["ReadAsset","asset10"]}'
```

```
(Naveen@LAPTOP-258I2DTL) [~/fabric-dev/fabric-samples/test-network]
$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
{"AppraisedValue":800,"Color":"Black","ID":"asset10","Owner":"Arun","Size":15}

(Naveen@LAPTOP-258I2DTL) [~/fabric-dev/fabric-samples/test-network]
$ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile "$ORDERER_CA" -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt" --peerAddresses localhost:9051 --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" -c '{"function":"TransferAsset","Args":["asset10","David"]}' --waitForEvent
2026-09-01 20:53:22.663 IST 0001 INFO [chaincodeCmd] ClientWait -> txid [4218c44c7466f4bfe5d1129519416ce5de3359a3d3cdd8d54f0951657201f343] committed with status (VALID) at localhost:9051
2026-09-01 20:53:22.663 IST 0002 INFO [chaincodeCmd] ClientWait -> txid [4218c44c7466f4bfe5d1129519416ce5de3359a3d3cdd8d54f0951657201f343] committed with status (VALID) at localhost:7051
2026-09-01 20:53:22.663 IST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Arun"

(Naveen@LAPTOP-258I2DTL) [~/fabric-dev/fabric-samples/test-network]
$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
{"AppraisedValue":800,"Color":"Black","ID":"asset10","Owner":"David","Size":15}
```

11. Stop the Network

```
./network.sh down
```

```
(Naveen@LAPTOP-258I2DTL) [~/fabric-dev/fabric-samples/test-network]
$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 10/10
✓Container ca_org1 Removed
✓Container ca_org2 Removed
✓Container orderer.example.com Removed
✓Container peer0.org1.example.com Removed
✓Container peer0.org2.example.com Removed
✓Container ca_orderer Removed
✓Network fabric_test Removed
✓Volume compose_orderer.example.com Removed
✓Volume compose_peer0.org1.example.com Removed
✓Volume compose_peer0.org2.example.com Removed
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Removing docker images
```

Result

The simple JavaScript chaincode was successfully developed, deployed, installed, approved, committed, and executed on the Hyperledger Fabric test network. Asset records were successfully created and queried, and ownership of **asset10** was successfully transferred from **Arun** to **David**, demonstrating the basic lifecycle of a smart contract in Hyperledger Fabric.